



**First semester 2022/2023**  
**Advanced topics in software development**  
**Homework Assignment # awxz3cdfv3e**  
**Event Processing System Refactoring**  
**ETA: 12 Dec midnight.**

**Create by : Ahmad istatieh 12217056**

**Under supervision**

Dr. Mustafa Assaf  
En.Mohammad Qzeh

## Before the Refactor How the System Works Now

### **We have a system called Event Processing System.**

This system is responsible for processing different types of events such as user actions , system related events , and security events.

Let's imagine a scenario where an event occurs in the system.

When an event is created, it contains a type "such as USER, SYSTEM, or SECURITY" , along with a payload that represents the event's data , The event may also contain several flags that indicate whether the payload should be encrypted , compressed , or enriched with additional metadata.

Once the event is ready, it is sent directly to the EventProcessor, which is the central component responsible for handling the entire event processing workflow , The EventProcessor starts by validating the event. It checks whether the payload exists and is not empty , If the payload is invalid, the event processing stops immediately.

If the event is valid, the EventProcessor then proceeds to notify other subsystems. It updates the system dashboard by sending the event to the Dashboard component , and it logs the event information using the Logger component.

After that, the EventProcessor performs multiple transformations on the event payload. Based on several boolean flags inside the Event object, the processor decides whether the payload should be encrypted, compressed, or combined with metadata. These transformations are executed sequentially using conditional statements inside the EventProcessor itself.

Once the payload transformation is complete, the EventProcessor generates a unique identifier for the event based on the current time and the processed payload. This ID is then stored inside the event object.

The processed data, along with the generated event ID, is then persisted by calling the Database component directly. The Database saves the event data immediately without using any abstraction or reuse mechanism.

Finally, the EventProcessor performs additional behavior based on the event type. It uses a series of conditional if/else statements to determine what action should be taken for each event type , for security events, the processor performs extra analysis and directly triggers a security alert , for user events, it executes user-specific logic , for system events, it logs system-related audit information.

All of these responsibilities including validation , transformation , notification , persistence , and type specific behavior are handled within the same EventProcessor class.

## Key Problems in This Code :

### Issue 1: Centralized Event Processing (Single Responsibility Violation) :

The main issue in the current implementation is the `EventProcessor.process()` method. This method handles all stages of event processing instead of delegating responsibilities to specialized components.

**Specifically, the method performs:** Event validation , Dashboard updates , Logging , Payload transformations , Event ID generation , Database persistence , Event type specific behavior

All of these operations are implemented inside a single method , which causes the logic to be tightly coupled and difficult to maintain. Any change in one processing step requires modifying the same method increasing the risk of errors as the system evolves.

can be seen in the following code :

```
public void process(Event e) {
    // validate the new event!
    if (e.getPayload() == null || e.getPayload().isEmpty()) {
        System.out.println("Invalid event");
        return;
    }

    // notify various subsystems with new event.
    dashboard.updateMetrics(e);
    logger.log(e);

    // transform (lots of if/else)
    String data = e.getPayload();
    if (e.isEncrypt()) {
        // naive encryption
        data = "ENC(" + data + ")";
    }
    if (e.isCompress()) {
        data = "CMP(" + data + ")";
    }
    if (e.isAddMetadata()) {
        data = "META(" + e.getMetadata() + ")::" + data;
    }

    // persist
    e.setId(System.currentTimeMillis() + "-" + Math.abs(data.hashCode()));
    database.save(e.getId(), data);

    // extra behavior depending on type
    if ("SECURITY".equals(e.getType())) {
        // duplicate security handling across code
        System.out.println("[SECURITY] extra analysis for " + e.getId());
        // directly send to security monitor (hard-coded)
        System.out.println("[SecurityMonitor] alert for " + e.getId());
    } else if ("USER".equals(e.getType())) {
        System.out.println("[USER] user-specific step for " + e.getId());
    } else if ("SYSTEM".equals(e.getType())) {
        System.out.println("[SYSTEM] system audit log " + e.getId());
    }
}
```

Because all responsibilities are centralized in one place, the system is hard to extend , difficult to test , and not well prepared for future changes such as adding new event types or processing steps.

## Issue 2: Event Type Handling with String and if/else (Poor Extensibility)

The current implementation handles event types using plain strings combined with a chain of if/else conditions inside EventProcessor.

The event type is stored as a String in the Event class :

```
private String type; // "USER","SYSTEM","SECURITY"
```

and is checked like this :

```
// extra behavior depending on type
if ("SECURITY".equals(e.getType())) {
    // duplicate security handling across code
    System.out.println("[SECURITY] extra analysis for " + e.getId());
    // directly send to security monitor (hard-coded)
    System.out.println("[SecurityMonitor] alert for " + e.getId());
} else if ("USER".equals(e.getType())) {
    System.out.println("[USER] user-specific step for " + e.getId());
} else if ("SYSTEM".equals(e.getType())) {
    System.out.println("[SYSTEM] system audit log " + e.getId());
}
```

This design cause two main problems:

### Error prone type representation

Using raw strings for event types makes the code fragile , Any typo in the string will not be detected at compile time and can break the logic silently.

### Hard to extend (Disagree Open/Closed Principle)

Adding a new event type requires editing the process() method and adding another else if branch , This means the core processing logic must be modified every time a new type is introduced , making the system difficult to extend and maintain.

Because of this design , the event type handling is tightly coupled to the EventProcessor implementation and does not scale well as the number of event types grows.

## Issue 3: Too Many Flags Inside Event (Feature Flags Controlling Logic Externally)

The Event class contains several boolean flags that control how the payload should be processed:

```
private boolean encrypt;  
private boolean compress;  
private boolean addMetadata;  
private String metadata;
```

These flags are then used inside `EventProcessor.process()` to decide which transformations to apply:

```
// transform (lots of if/else)  
String data = e.getPayload();  
if (e.isEncrypt()) {  
    // naive encryption  
    data = "ENC(" + data + ")";  
}  
if (e.isCompress()) {  
    data = "CMP(" + data + ")";  
}  
if (e.isAddMetadata()) {  
    data = "META(" + e.getMetadata() + "):" + data;  
}
```

This design introduces the following problems:

### **Growing complexity with every new feature**

Each new processing option will require :

A new flag in the Event class.

New if conditions in EventProcessor.

Over time, this turns the processing code into a long list of conditionals that are hard to manage.

### **Mixing data with processing decisions**

The Event object is supposed to represent the event data (type, payload, id) , but it also stores processing decisions (encrypt, compress, addMetadata) , This blurs the separation between what the event is and how it should be processed, making both the Event and EventProcessor more complicated.

Because of these flags, the processing logic becomes scattered and harder to extend , Adding or changing transformations always requires modifying the core processing method and the Event structure at the same time.

## Issue 4: Tight Coupling to Concrete Components (Database, Dashboard, Logger, Security Logic)

The EventProcessor directly depends on concrete classes instead of abstractions :

```
private Database database;  
private Dashboard dashboard;  
private Logger logger;
```

It also contains hard coded security behavior :

```
// duplicate security handling across code  
System.out.println("[SECURITY] extra analysis for " + e.getId());  
// directly send to security monitor (hard-coded)  
System.out.println("[SecurityMonitor] alert for " + e.getId());
```

This causes two main problems:

### Hard to change or replace dependencies

If the system later needs to:

Replace Database with another storage mechanism , Change how logging works , Or move security alerts to another module

then the EventProcessor itself must be modified , This increases coupling and makes the class sensitive to changes in other parts of the system.

### Difficult to test and mock

Because EventProcessor is directly tied to real implementations, it is harder to unit test in isolation. You cannot easily swap in test doubles or mock components without changing the class.

This tight coupling reduces flexibility and makes the overall architecture less maintainable as the system evolves.

## After the Refactor How the System Works Now

After the refactor, the Event Processing System no longer relies on a single, complex method inside one class. Instead, the system is split into smaller, focused components that collaborate to process each event.

When a new event is needed, the system can create it by cloning an existing event prototype. For example, a predefined template event with a specific type and default payload is cloned using a new identifier. This use of the Prototype pattern avoids rebuilding the configuration from scratch every time and keeps event creation simple and consistent.

Once the event is created, it is passed to the event processing pipeline through the `IEventProcessor` interface. In most cases, the call first goes through the `EventProcessorProxy`. The proxy is responsible for cross-cutting concerns: it logs that processing has started, checks access rules (for example, it can block certain security events), and measures how long the processing takes. If the proxy allows the event, it forwards the request to the real `EventProcessor`.

Inside the `EventProcessor`, the flow is much clearer and more focused. The processor first validates the event payload; if the payload is missing or empty, it logs an error and stops the workflow early. For valid events, the processor then applies a sequence of transformation strategies to the payload. These transformations are modeled using the Strategy pattern: each `EventProcessingStrategy` (such as encryption or adding metadata) is a separate class, and the processor simply iterates over a list of strategies and applies them one by one. No boolean flags are stored inside the event, and no long chains of if/else statements are needed.

After the payload has been transformed, the processed data and the event identifier are persisted through the `IEventDatabase` abstraction. The concrete implementation, `SqlEventDatabase`, does not open a new database connection for every request. Instead, it uses a `ConnectionPool` to acquire a reusable connection, execute the save operation, and then release the connection back to the pool. This Connection Pool pattern improves performance and keeps resource usage under control.

Once the event has been stored, the `EventProcessor` notifies other parts of the system. It updates the Dashboard component with the final processed data, and uses the `Logger` interface to record a structured log entry for the event.

This notification mechanism is implemented using the Observer pattern, where both the Dashboard and the Logger subscribe to event updates. The logging behavior itself is extended using the Decorator pattern `TimeLogger`, which adds timestamps without changing the core logger.

Finally, any behavior that depends on the event type is handled using a separate Strategy layer. The event type is now represented as a strong `EventType` enum instead of a raw string. The `EventTypeStrategyFactory` returns the appropriate handler for each type (`USER`, `SYSTEM`, `SECURITY`), and the selected strategy executes its own logic for that event. This removes the need for conditional if/else chains inside the processor and makes it easy to add new event types without changing the core processing code.

## **How Each Problem Was Solved :**

### **Issue 1: Rebuilding Event Configuration Repeatedly (Solved with Prototype Pattern)**

In the refactored version, event configurations are no longer rebuilt from scratch every time a new event is created.

Instead, the Prototype Pattern is used to provide predefined event templates that can be cloned quickly and safely.

Each event type (USER, SECURITY, SYSTEM) can now have a preset configuration stored once.

When a new event is processed, the system simply clones the template and assigns it a new identifier, ensuring fast, consistent configuration.

This eliminates duplicated setup logic, reduces errors, and makes the event creation process more efficient and predictable.

Example (fixed code):

```
Event e1 = template.cloneWithId( newId: 2);
```

Issue 2: No Real Connection Pool (Solved with Connection Pool Pattern) :

After refactoring, a Connection Pool was introduced.

The pool maintains a fixed number of reusable database connections that are: acquired when needed , used to execute the database operation , safely released back into the pool

This prevents excessive connection creation and ensures controlled, efficient resource usage.

Example (fixed code):

```
@Override 1 usage new *
public void save(int id, String data) {
    Connection c = pool.acquire();
    try {
        c.execute( cmd: "INSERT events(id,data) VALUES(" + id + "," + data + ")");
        System.out.println("[DB] saved " + id);
    } finally {
        pool.release(c);
    }
}
```

Issue 3: Cross Cutting Concerns Mixed with Core Processing (Solved with Proxy Pattern)

Originally, the EventProcessor was responsible not only for the main business logic, but also for cross-cutting concerns such as access checks, logging, and measuring execution time.

This made the core processing method harder to read, test, and extend.

After refactoring, the Proxy pattern was applied by introducing the EventProcessorProxy.



The proxy now acts as an intermediate layer that : logs when processing starts and finishes , checks access rules (for example, blocks security events) , measures execution time , forwards the request to the real EventProcessor

This cleanly separates cross-cutting concerns from the core processing logic and keeps the processor simple.

Example (fixed code):

```
EventProcessorInter processor = new EventProcessorProxy(realProcessor, logger);
```

Issue 4: Long Conditional Logic for Event Types (Solved with Strategy Pattern) :

In the old design , the processor used long if/else chains to handle different event types. This made the system: hard to extend , hard to maintain , error-prone , violating the Open/Closed Principle

In the new design, each event type now has a dedicated handler implemented using the Strategy Pattern.

A factory EventTypeStrategyFactory dynamically selects the correct strategy at runtime. This completely removes conditional logic and makes adding new event types extremely simple just create a new strategy class.

Example (fixed code):

```
typeFactory.get(e.getType()).handle(e, logger);
```

## Design Decisions

While working on refactoring the Event Processing System, the main goal was to clean up the code and make it easier to understand and extend in the future.

## Overall Design Approach

In the original version , most of the work was done inside one class. That class was responsible for processing events, deciding how each event should behave , and dealing with the database , Because everything was mixed together, the code became hard to read and risky to change.

To improve this , the logic was divided into smaller , focused classes. Each part of the system now has a clear responsibility , which makes the code easier to follow and simpler to modify later if needed.

## **Using Prototype Pattern**

Event configurations used to be decided manually every time an event was processed. Instead of repeating the same setup logic, predefined event profiles are now created once and stored in a registry.

When a new event arrives, the system simply clones the related profile instead of creating it from scratch , This made the configuration process cleaner and more consistent.

This pattern was chosen instead of manually building each event in different places , because it keeps the configuration in one reusable template.

Another option was using constructors with many parameters , but that would make the code harder to read and easier to misuse.

## **Using Strategy Pattern**

Different event types require different behavior.

Handling this using if/else statements would quickly make the code messy as more event types are added.

To avoid this , each event type has its own handler , The system selects the correct handler at runtime, keeping the processing logic simple and organized.

The Strategy pattern was chosen instead of long if/else or switch statements , because it lets us add new event types without changing the core processor.

This keeps the design open for extension and closed for modification.

## **Using Proxy and Connection Pool Patterns**

Direct interaction with the database was avoided to keep the processing logic clean.

A proxy was added to control access to the database and handle connection management.

A connection pool was also introduced so database connections can be reused instead of created every time , This approach makes the system more efficient and easier to control.

these design decisions make the system Easier to read , Easier to extend , Better organized , More efficient in handling resources.

The Proxy pattern was chosen to move cross cutting concerns like access checks and logging outside the main processor , instead of mixing everything in one class.

The Connection Pool pattern was selected instead of creating a new database connection every time , which would not scale well when the system grows.

## Using Observer Pattern

The Observer Pattern was introduced to decouple event notifications from the processing logic. Instead of the processor directly updating the dashboard or logger, it now sends a notification through EventSubject, which forwards the event to all registered observers.

This makes the system more flexible, since new observers can be added without modifying the EventProcessor.

An alternative would be to call each component directly from the processor, but that would make the code tightly coupled.

With the Observer pattern, new listeners can be added or removed without changing the event processing logic.

## Using Decorator Pattern

The logging mechanism was extended using the Decorator Pattern.

Instead of modifying the original logger to add timestamps or extra formatting, a TimeLogger decorator wraps the base logger and adds new behavior dynamically.

This keeps the logger design open for extension but closed for modification.

Without the Decorator pattern we would need many different logger subclasses for each extra feature, which would make the design harder to manage.

Using decorators keeps the base logger simple while still allowing flexible combinations of behavior.

## Using Template Method Pattern

Different event types require different handling logic.

To avoid repeating common steps (start log, end log) across handlers, the Template Method Pattern was applied using AbstractEventTypeHandler.

Each specific handler (USER, SYSTEM, SECURITY) only implements the custom part, while the template handles the shared steps, improving code reuse and consistency.

This pattern was chosen to avoid copy pasting the same start and end steps in every handler. It also makes sure that all event types follow the same high level flow while still allowing custom logic for each type.

## Patterns Interaction and Benefits

All of these patterns work together, The Proxy controls access before calling the EventProcessor, the processor uses strategies and the template method to handle different

event types , the observer notifies the dashboard and logger , the decorator extends logging behavior , and the connection pool saves data efficiently.

This combination makes the architecture more scalable , more flexible , and easier to maintain in the future.

### The output of the Event Processing System :

```
--- Processing Event 1 | Type: USER ---
Step 1: Proxy check / security filtering
[22:31:45.089738] [Proxy] checking 1
Step 2: Notifying observers
[EventSubject] Notifying 2 observers for Event 1
[22:31:45.096318] [Dashboard] metrics updated for 1, payload: user-click
[LOGGER] event Id: 1 event Type: USER
Step 3: Applying strategies
[Strategy] Applied AddMetadataStrategy -> payload: META(user-42)::user-click
[Strategy] Applied EncryptStrategy -> payload: ENC(META(user-42)::user-click)
[Conn C1] INSERT events(id,data) VALUES(1,ENC(META(user-42)::user-click))
[DB] saved 1
Step 4: Executing event-type strategy
[22:31:45.097741] [Handler] start handling type: USER
[22:31:45.097941] [USER] user audit log 1
[22:31:45.098029] [Handler] finished handling type: USER
[22:31:45.102542] [Proxy] finished 1 in 5.001676 ms
--- Finished Event 1 ---

--- Processing Event 2 | Type: SYSTEM ---
Step 1: Proxy check / security filtering
[22:31:45.102813] [Proxy] checking 2
Step 2: Notifying observers
[EventSubject] Notifying 2 observers for Event 2
[22:31:45.102962] [Dashboard] metrics updated for 2, payload: system-update
[LOGGER] event Id: 2 event Type: SYSTEM
Step 3: Applying strategies
[Strategy] Applied AddMetadataStrategy -> payload: META(user-42)::system-update
[Strategy] Applied EncryptStrategy -> payload: ENC(META(user-42)::system-update)
[Conn C2] INSERT events(id,data) VALUES(2,ENC(META(user-42)::system-update))
[DB] saved 2
Step 4: Executing event-type strategy
[22:31:45.103208] [Handler] start handling type: SYSTEM
[22:31:45.103526] [SYSTEM] system audit log 2
[22:31:45.103591] [Handler] finished handling type: SYSTEM
[22:31:45.103643] [Proxy] finished 2 in 0.729318 ms
--- Finished Event 2 ---
```

**In the end :** refactoring the Event Processing System improved its structure and flexibility ,  
Using well known design patterns helped separate responsibilities and made the system easier  
to maintain and extend in the future.